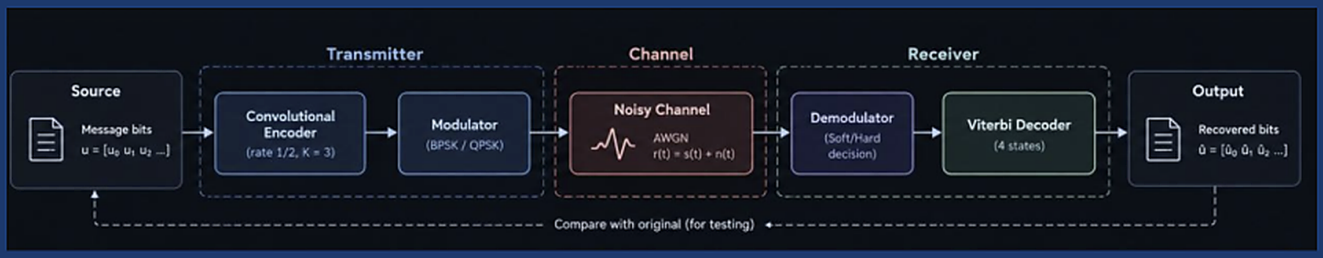
INITIAL PROGRESS REPORT

Convolutional Encoder • Noisy Channel • Viterbi Decoder

Group No: 18

Date: 12 October 2026

1. Updated Block Diagram



Overall flow: Input Bits → Convolutional Encoder → Noisy Channel → Viterbi Decoder → Recovered Bits

2. Initial Implementation

At this stage, we have worked individually on their assigned modules. Each member is checking their portion for internal errors before the modules are connected. Once completed, we'll have a common program running all the modules together. Also, we have used separate testbench cases for now.

A.) Encoder

I implemented the convolutional encoder in Verilog using XOR logic and two memory bits to produce encoded outputs from the input data. I also created a test bench that generated the clock and reset signals, applied different input bit sequences, and monitored the resulting outputs. Using EDA Playground, I simulated the design and verified its operation, including how previous input bits influence each output.

testbench.sv

```
`timescale 1ns/1ps

module tb_conv_encoder;

    reg clk, rst;
    reg input_bit, input_valid;
  wire [1:0] encoded_pair;
    wire output_valid;

    integer failures;

    conv_encoder dut (
        .clk(clk),
        .rst(rst),
      .input_bit(input_bit),
      .input_valid(input_valid),
      .encoded_pair(encoded_pair),
      .output_valid(output_valid)
    );

    always #5 clk = ~clk;

    task test_bit;
        input data;
        input [1:0] expected;
        begin
            @(negedge clk);
            input_bit = data;

            @(posedge clk);
            #1;

            $display("Input = %b, Output = %b, Expected = %b",
                     input_bit, encoded_pair, expected);

          if (encoded_pair !== expected)
              failures = failures + 1;
        end
    endtask

    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, tb_conv_encoder);

        clk = 0;
        rst = 1;
        input_bit = 0;
        input_valid = 0;
        failures = 0;

        #12 rst = 0;
        input_valid = 1;

        test_bit(1, 2'b11);
        test_bit(0, 2'b10);
        test_bit(1, 2'b00);
        test_bit(1, 2'b01);
        test_bit(0, 2'b01);
        test_bit(0, 2'b11);

        @(negedge clk);
        input_valid = 0;

        @(posedge clk);
        #1;

      if (output_valid !== 0)
            failures = failures + 1;

        if (failures == 0)
            $display("TEST PASSED");
        else
            $display("TEST FAILED: %0d errors", failures);

        $finish;
    end

endmodule
```

design.sv

```
module conv_encoder(
    input wire clk,
    input wire rst,
    input wire input_bit,
    input wire input_valid,
  output reg [1:0] encoded_pair,
    output reg output_valid
);

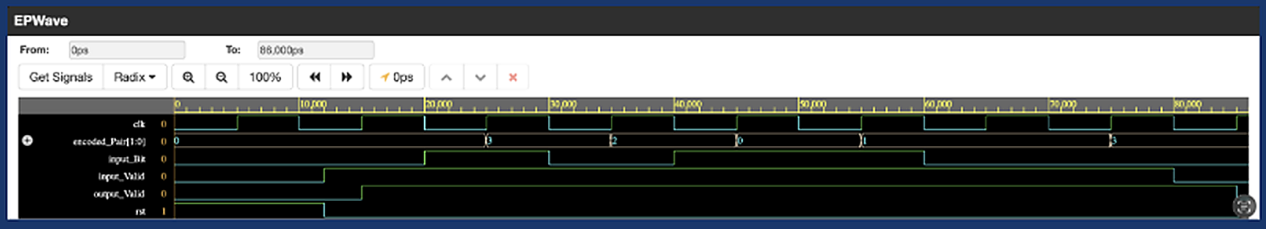
    reg s1;
    reg s2;

always @(posedge clk or posedge rst) begin
    if (rst)
      begin
        s1<= 1'b0;
        s2 <= 1'b0;
        encoded_pair <= 2'b00;
        output_valid <= 1'b0;
    end
    else begin
      output_valid <= input_valid;

 if (input_Valid)
 begin
    encoded_pair[1] <= input_bit ^ s1 ^ s2;
    encoded_pair[0] <= input_bit ^ s2;
     s1 <= input_bit;
     s2 <= s1;
    end
        end
    end

endmodule
```

```
Input = 1, Output = 11, Expected = 11
Input = 0, Output = 10, Expected = 10
Input = 1, Output = 00, Expected = 00
Input = 1, Output = 01, Expected = 01
Input = 0, Output = 01, Expected = 01
Input = 0, Output = 11, Expected = 11
TEST PASSED
testbench.sv:75: $finish called at 86000 (1ps)
```



B.) Noisy Channel

My part of the project is the noisy channel, which simulates transmission errors in our encoded data. The design receives a 2-bit encoded pair and an error mask. First, it checks whether error injection is enabled. If it is disabled, the active mask becomes zero, so the encoded data passes through unchanged. If it is enabled, the supplied error mask is applied using the XOR operation, which flips the selected bits. The channel also uses the input-valid signal to determine whether the incoming data should be processed. All outputs are updated synchronously on the rising edge of the clock, and reset initializes the outputs. To verify the design, I created a testbench. It generates the clock, applies reset, and sends different input

combinations to the design. For every test case, it calculates the expected result independently and compares it with the actual output. If the outputs match, the test passes; otherwise, the simulation stops with a failure message. Finally, the testbench reports the number of successful test cases. This allows us to verify that both normal transmission and controlled error injection behave as expected.

design.sv

```
module noisy_channel (
    input  wire       clk,
    input  wire       rst,
    input  wire       input_valid,
    input  wire [1:0] encoded_pair,
    input  wire       error_enable,
    input  wire [1:0] error_mask,

    output reg        output_valid,
    output reg  [1:0] received_pair
);

    wire [1:0] active_mask;

    assign active_mask = error_enable
                       ? error_mask : 2'b00;

    always @(posedge clk) begin
        if (rst) begin
            output_valid <= 1'b0;
            received_pair <= 2'b00;
        end else begin
            output_valid <= input_valid;

            if (input_valid)
                received_pair <= encoded_pair ^ active_mask;
            else
                received_pair <= 2'b00;
        end
    end

endmodule
```

testbench.sv

```
`timescale 1ns/1ps

module testbench;

    reg clk;
    reg rst;
    reg input_valid;
    reg [1:0] encoded_pair;
    reg error_enable;
    reg [1:0] error_mask;

    wire output_valid;
    wire [1:0] received_pair;

    integer tests_passed;

    noisy_channel dut (
        .clk(clk),
        .rst(rst),
        .input_valid(input_valid),
        .encoded_pair(encoded_pair),
        .error_enable(error_enable),
        .error_mask(error_mask),
        .output_valid(output_valid),
        .received_pair(received_pair)
    );

    initial clk = 0;
    always #5 clk = ~clk;

    task check_case;
        input valid_value;
        input [1:0] pair_value;
        input enable_value;
        input [1:0] mask_value;
        reg [1:0] expected_pair;
        begin
            @(negedge clk);
            input_valid = valid_value;
            encoded_pair = pair_value;
            error_enable = enable_value;
            error_mask = mask_value;

            @(posedge clk);
            #1;

            expected_pair = pair_value ^
                (enable_value ? mask_value : 2'b00);

            if (output_valid !== valid_value) begin
                $display("FAIL: valid signal mismatch");
                $fatal(1);
            end

            if (valid_value &&
                received_pair !== expected_pair) begin
                $display("FAIL: input=%b mask=%b expected=%b got=%b",
                    pair_value, mask_value,
                    expected_pair, received_pair);
                $fatal(1);
            end

            tests_passed = tests_passed + 1;

            $display("PASS %0d: input=%b enable=%b mask=%b received=%b",
                tests_passed, pair_value, enable_value,
                mask_value, received_pair);
        end
    endtask

    initial begin
        $dumpfile("noisy_channel.vcd");
        $dumpvars(0, testbench);

        rst = 1;
        input_valid = 0;
        encoded_pair = 0;
        error_enable = 0;
        error_mask = 0;
        tests_passed = 0;

        repeat (2) @(posedge clk);
        @(negedge clk);
        rst = 0;

        // Clean channel
        check_case(1, 2'b00, 0, 2'b00);
        check_case(1, 2'b01, 0, 2'b11);
        check_case(1, 2'b10, 0, 2'b01);
        check_case(1, 2'b11, 0, 2'b10);

        // Single-bit errors
        check_case(1, 2'b00, 1, 2'b01);
        check_case(1, 2'b11, 1, 2'b10);

        // Double-bit errors
        check_case(1, 2'b00, 1, 2'b11);
        check_case(1, 2'b10, 1, 2'b11);

        // Error disabled: mask must be ignored
        check_case(1, 2'b10, 0, 2'b11);

        // Invalid input
        check_case(0, 2'b11, 1, 2'b11);

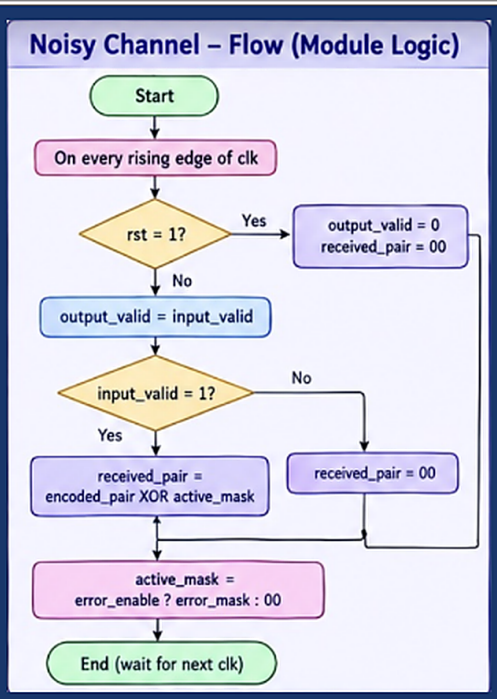
        $display("--------------------------------");
        $display("TEST PASSED: %0d cases checked.",
                 tests_passed);
        $display("--------------------------------");

        #10;
        $finish;
    end

endmodule
```

Log | Share

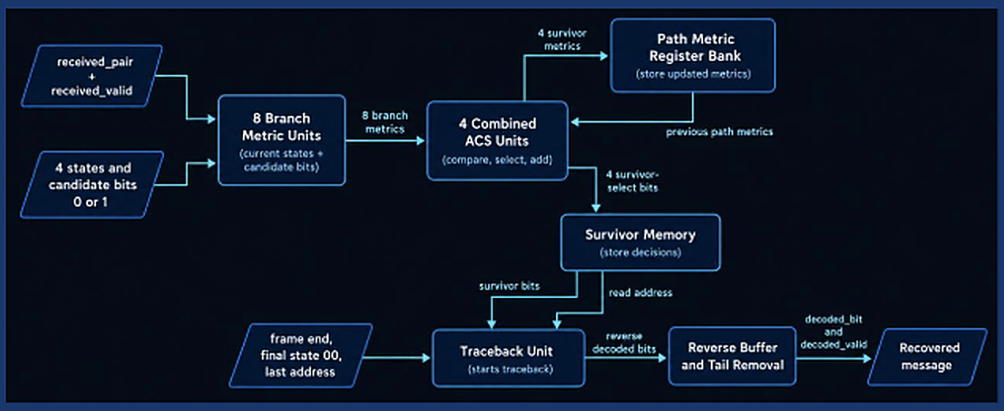
```
PASS 1: input=00 enable=0 mask=00 received=00
PASS 2: input=01 enable=0 mask=11 received=01
PASS 3: input=10 enable=0 mask=01 received=10
PASS 4: input=11 enable=0 mask=10 received=11
PASS 5: input=00 enable=1 mask=01 received=01
PASS 6: input=11 enable=1 mask=10 received=01
PASS 7: input=00 enable=1 mask=11 received=11
PASS 8: input=10 enable=1 mask=11 received=01
PASS 9: input=10 enable=0 mask=11 received=10
PASS 10: input=11 enable=1 mask=11 received=00
--------------------------------
TEST PASSED: 10 cases checked.
```



B.) Viterbi Decoder

Our Viterbi decoder is a hard-decision error-correction system that receives noisy encoded bit pairs and finds the most likely original message sequence. It is designed for the rate 1/2, constraint-length K = 3 convolutional encoder used in our project. The decoder is divided into five main modules: the Branch Metric Unit, which compares each possible encoder output with the received pair; the combined ACS Unit, which adds path errors, compares competing routes, and selects the better route; the Path Metric Register Bank, which stores the best error value for each trellis state; Survivor Memory, which stores the winning route decisions; and the Traceback Unit, which follows those stored decisions backward to recover the original data bits.

The data flow starts when a noisy encoded pair enters the Branch Metric Units. Their error values and the previously stored path metrics are processed by the ACS units. The selected metrics are saved for the next received pair, while the selected route choices are written into Survivor Memory. After the full message and tail bits are received, the Traceback Unit starts from the known final state 00, reads the survivor decisions in reverse order, and reconstructs the most likely transmitted message.




```
1. Branch Metric Unit

module viterbi_branch_metric (
    input wire [1:0] current_state,
    input wire candidate_bit,
    input wire [1:0] received_pair,
    output reg [1:0] expected_pair,
    output reg [1:0] next_state,
    output reg [1:0] branch_metric
);

    always @(*) begin
        expected_pair[1] = candidate_bit ^ current_state[1] ^ current_state[0];
        expected_pair[0] = candidate_bit ^ current_state[0];
        next_state = {candidate_bit, current_state[1]};
        branch_metric = {1'b0, received_pair[1] ^ expected_pair[1]} +
                        {1'b0, received_pair[0] ^ expected_pair[0]};
    end
endmodule
```

```
2. ACS and Path Metric Adder

module viterbi_acs_unit #(
    parameter METRIC_W = 8
) (
    input wire [METRIC_W-1:0] previous_metric_a,
    input wire [METRIC_W-1:0] previous_metric_b,
    input wire [1:0] branch_metric_a,
    input wire [1:0] branch_metric_b,

    output wire [METRIC_W-1:0] candidate_metric_a,
    output wire [METRIC_W-1:0] candidate_metric_b,
    output reg [METRIC_W-1:0] survivor_metric,
    output reg survivor_select
);

    assign candidate_metric_a =
        previous_metric_a + {{(METRIC_W-2){1'b0}}, branch_metric_a};

    assign candidate_metric_b =
        previous_metric_b + {{(METRIC_W-2){1'b0}}, branch_metric_b};

    always @(*) begin
        if (candidate_metric_a <= candidate_metric_b) begin
            survivor_metric = candidate_metric_a;
            survivor_select = 1'b0;
        end else begin
            survivor_metric = candidate_metric_b;
            survivor_select = 1'b1;
        end
    end
endmodule
```

```
5. TraceBack Unit

module traceback_unit #(
    parameter ADDR_W = 4
) (
    input wire clk,
    input wire rst,
    input wire start,
    input wire [ADDR_W-1:0] last_address,
    input wire [ADDR_W:0] traceback_length,
    input wire [1:0] final_state,
    input wire [3:0] survivor_bits,
    output reg [ADDR_W-1:0] read_address,
    output reg decoded_bit,
    output reg decoded_valid,
    output reg busy,
    output reg done
);

    reg [1:0] current_state;
    reg [ADDR_W:0] remaining_steps;

    always @(posedge clk) begin
        if (rst) begin
            read_address <= {ADDR_W{1'b0}};
            decoded_bit <= 1'b0;
            decoded_valid <= 1'b0;
            current_state <= 2'b00;
            remaining_steps <= {(ADDR_W+1){1'b0}};
            busy <= 1'b0;
            done <= 1'b0;
        end else begin
            decoded_valid <= 1'b0;
            done <= 1'b0;

            if (start && !busy) begin
                read_address <= last_address;
                current_state <= final_state;
                remaining_steps <= traceback_length;
                busy <= 1'b1;
            end else if (busy && remaining_steps != 0) begin
                decoded_bit <= current_state[1];
                decoded_valid <= 1'b1;
                current_state <= {current_state[0], survivor_bits[current_state]};
                read_address <= read_address - 1'b1;
                remaining_steps <= remaining_steps - 1'b1;

                if (remaining_steps == 1) begin
                    busy <= 1'b0;
                    done <= 1'b1;
                end
            end
        end
    end
```

```
3. Path Metric Register Bank

module path_metric_bank #(
    parameter METRIC_W = 8,
    parameter [METRIC_W-1:0] INF_METRIC = 8'd63
) (
    input wire clk,
    input wire rst,
    input wire update_enable,
    input wire [METRIC_W-1:0] next_metric_00,
    input wire [METRIC_W-1:0] next_metric_01,
    input wire [METRIC_W-1:0] next_metric_10,
    input wire [METRIC_W-1:0] next_metric_11,
    output reg [METRIC_W-1:0] metric_00,
    output reg [METRIC_W-1:0] metric_01,
    output reg [METRIC_W-1:0] metric_10,
    output reg [METRIC_W-1:0] metric_11
);

    always @(posedge clk) begin
        if (rst) begin
            metric_00 <= {METRIC_W{1'b0}};
            metric_01 <= INF_METRIC;
            metric_10 <= INF_METRIC;
            metric_11 <= INF_METRIC;
        end else if (update_enable) begin
            metric_00 <= next_metric_00;
            metric_01 <= next_metric_01;
            metric_10 <= next_metric_10;
            metric_11 <= next_metric_11;
        end
    end

endmodule
```

```
4. Survivor Memory

module survivor_memory #(
    parameter TRACE_DEPTH = 16,
    parameter ADDR_W = 4
) (
    input wire clk,
    input wire rst,
    input wire write_enable,
    input wire [3:0] survivor_selects,
    input wire [ADDR_W-1:0] read_address,
    output wire [3:0] survivor_bits,
    output reg [ADDR_W-1:0] write_pointer
);

    reg [3:0] memory [0:TRACE_DEPTH-1];

    always @(posedge clk) begin
        if (rst) begin
            write_pointer <= {ADDR_W{1'b0}};
        end else if (write_enable) begin
            memory[write_pointer] <= survivor_selects;
            write_pointer <= write_pointer + 1'b1;
        end
    end

    assign survivor_bits = memory[read_address];

endmodule
```

3. Design Methodology

- Each module is developed and tested independently before integration.
- EDA Playground will be used for HDL coding and initial simulation.
- EPWave will be used to inspect simulation waveforms where supported.
- Git/GitHub will be used to manage code versions and combine the modules.
- The same test cases will be used during integration to compare expected and actual outputs.

4. Integration Plan

Software Integration — This Week

- Connect the encoder, noisy channel, and Viterbi decoder in one top-level design.
- Align input/output widths, bit ordering, clock, reset, and signal timing.
- Run clean-channel tests first, then test controlled bit errors.
- Compare the original input with the decoded output and record results.

Hardware Integration — Next Week

- Confirm the available FPGA board and select its compatible toolchain.
- Use AMD Vivado for supported AMD/Xilinx boards or Intel Quartus Prime for supported Intel/Altera boards.
- Synthesize the design, assign pins, check timing, and program the FPGA.
- Test known input sequences using onboard switches or a suitable input interface.
- Observe outputs using LEDs, UART/USB serial output, or available lab instruments.
- Compare hardware behavior with the software simulation results.

Software and Hardware Required

Type	Tools / Equipment	Purpose
Software	EDA Playground	HDL coding and simulation
Software	EPWave	Waveform inspection
Software	Git / GitHub	Version control and team integration
Software	Vivado or Quartus Prime	FPGA synthesis and programming; select after confirming the board
Hardware	FPGA development board	Run the integrated design
Hardware	USB programming cable and board power supply	Program and power the board
Hardware	Switches / LEDs or UART interface	Provide test inputs and observe outputs
Hardware	Optional logic analyzer / oscilloscope	Debug digital signals if available

5. Team Contributions

Team Member	Assigned Part	Work Completed
Krrish	Encoder	Individual Logic design and testing
Arya	Noisy Channel	Individual Logic design and testing
Gorish	Viterbi Decoder	Individual Logic design and testing
Maninder	Integration of Blocks (software +hardware)	Online connections and platform, hardware research for next week.
Maniyam	Documentation and testing	

CS203 – WEEK 3 – PROJECT UPDATE

Convolution Encoder– Noisy channel – Viterbi Decoder

Group no: 18

What we have done this week :

During this week, we successfully connected the convolutional encoder, noisy channel, and Viterbi decoder to develop the main communication pipeline. We tested manual test cases to verify the functioning of the connected modules and completed the integration of the error-generation code, which introduces errors into the transmitted encoded data. Now we have made a fully automated testbench files which generate random testcases and display the outcome. This work represents an important step toward testing the complete system and evaluating the decoder's ability to recover the original input data from corrupted transmissions.

Challenges faced :

- Integratiing logic files and connecting different modules & simplified the connections by lining up the input and output parameters for each module
- Designing a common test bench file which also provides bits data of each stage
- Fully automating the test cases by random generation . half of the tes t cases are error free and rest are injected with error .
- Saving logs and csv file has auto output.

Literature :

GIT HUB:

https://github.com/GLK769/DLD-

Research papers:

1. Convolutional Encoder

Research Paper: "Implementation of Convolutional Encoder and Viterbi Decoder using VHDL"

https://www.researchgate.net/publication/224129417_Implementation_of_Convolutional_Encoder_and_Viterbi_Decoder_using_VHDL

Contribution to Our Project: Helped us understand the working of a convolutional encoder and its use of XOR operations and memory bits, which relates to our Verilog encoder module.

2. Viterbi Decoder

Research Paper: "FPGA Design and Implementation of a Convolutional Encoder and a Viterbi Decoder Based on 802.11a for OFDM" (https://doi.org/10.4236/WET.2012.33019)

https://doi.org/10.4236/WET.2012.33019

Contribution to Our Project: Helped us understand branch metrics, path metrics, and ACS operations used in our Viterbi decoder.

3. Survivor Memory and Traceback

Research Paper: "Architectural Tradeoffs for Survivor Sequence Memory Management in Viterbi Decoders" (https://ieeexplore.ieee.org/abstract/document/221067)

Contribution to Our Project: Helped us understand how survivor decisions are stored and used by our decoder's traceback module to reconstruct the input sequence.

4. Noisy Channel

Research Paper: "Error Performance Analysis of Forward Error Correction Using Convolutional Encoding in the Presence of (1/f) Noise" (https://beei.org/index.php/EEI/article/view/4771)

Contribution to Our Project: Helped us understand how channel errors can affect transmitted data and why our noisy-channel module introduces controlled bit errors to test the decoder.

5. Encoder and Decoder Implementation

Research Paper: "FPGA Implementation of High Speed Convolutional Encoder and Viterbi Decoder for Software Defined Radio" (https://ieeexplore.ieee.org/abstract/document/10179840)

Contribution to Our Project: Helped us understand HDL-based implementation and testing of convolutional encoders and Viterbi decoders, which is relevant to our Verilog-based design.

6. EDA Playground

Technical Documentation: "EDA Playground Documentation" (https://github.com/edaplayground/eda-playground/blob/master/docs/intro.rst)

Contribution to Our Project: Helped us understand how to run and simulate our Verilog modules and test their functionality using EDA Playground.

Technical documentations

7. Waveform Analysis

Technical Documentation: "EPWave Documentation"

https://epwave.readthedocs.io/en/latest/quick-start.html

Contribution to Our Project: Helped us understand how to inspect simulation waveforms and observe signal behaviour while debugging and verifying our modules.

Next week target :

Next week, we will extend the initial working communication link into an automated performance-evaluation system. The testbench will transmit many random four-bit data frames through both error-free and controlled noisy-channel conditions. For each frame, the system will record the transmitted payload, selected error location, received data, decoded data, channel bit errors, payload bit errors, and PASS/FAIL status. This will allow us to calculate Channel BER, Decoded BER, frame success rate, and the error-correction effectiveness of the Viterbi decoder.

We will also improve presentation quality by adding a trellis-path explanation, waveform evidence for both clean and corrupted transmission, and a comparison table between raw channel errors and final decoded errors. The objective is to demonstrate that the decoder does not merely pass data forward: it uses branch metrics, ACS decisions, survivor memory, and traceback to select the most likely original message.

Planned deliverables:

- Automated multi-frame simulation with random payloads.
- Separate results for no-error and one-bit-error conditions.
- CSV result log and summarized performance table.
- BER and frame-success calculations.
- EPWave screenshots showing a corrupted encoded pair being correctly decoded.
- Final flowchart and Viterbi trellis explanation for the report/presentation.

Contribution:

ARYA : Literature and source collection

GORISH & KRRISH : Logic building and module integration (design.sv)

MAINAM & MAANINDER : TEST cases designing , fully automation and testing of the system , documentation.